

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



A Visualized Intro to Git Internals — Objects and Branches



Omer Rosenbaum · Follow

Published in Swimm

13 min read · Apr 27, 2020



Listen



Share



More

Many of us use `git` on a daily basis. But how many of us know what goes on under the hood? For example, what happens when we use `git commit`? What is stored between commits? Is it just a diff between the current and previous commit? If so, how is the diff encoded? Or is an entire snapshot of the repo stored each time? What really happens when we use `git init`?

Many people who use `git` don't know the answers to the questions above. But does it really matter? First, as professionals, we should strive to understand the tools we use, especially if we use them all the time — like `git`. But even more acutely, I found that understanding how `git` actually works is useful in many scenarios — whether it's **merge conflicts**, looking to conduct an interesting rebase, or even just when something goes slightly wrong.

This is the first post in a series discussing `git` mechanisms and is part of our [“Swimminars”](#) blog series. You'll benefit from this post if you're experienced enough with `git` to feel comfortable with commands such as `git pull`, `git push`, `git add` or `git commit`. Yet, we will start with an overview to make sure we are on the same page regarding the mechanisms of `git`, and specifically, the terms used throughout this post.

Update: I uploaded a YouTube series covering this post — you are welcome to watch it [here](#).

Note:  My book “Gitting Things Done” is officially out!

Get it [on Amazon](#), as [an eBook](#), Or read it for free at:

<https://www.freecodecamp.org/news/gitting-things-done-book/>

What to Expect

We will get a rare understanding of what goes on under the hood of what we do almost daily. We will start by covering objects — **blobs**, **trees**, and **commits**. We will then briefly discuss **branches** and how they are implemented. We will dive into the **working directory**, **staging area** and **repository**. We will make sure we understand how these terms relate to `git` commands we know and use to create a new repository.

On [the next post](#), we will create a repository from scratch — without using `git init`, `git add`, or `git commit`. This will allow us to **deepen our understanding of what is happening under the hood** when we work with `git`. We will also create new branches, switch branches and create additional commits — all without using `git branch` or `git checkout`.

By the end of these two posts, you will feel like you *understand* `git`. Are you up for it? 😎

Git Objects — blob, tree and commit

It is very useful to think about `git` as maintaining a file system, and specifically — snapshots of that system in time.

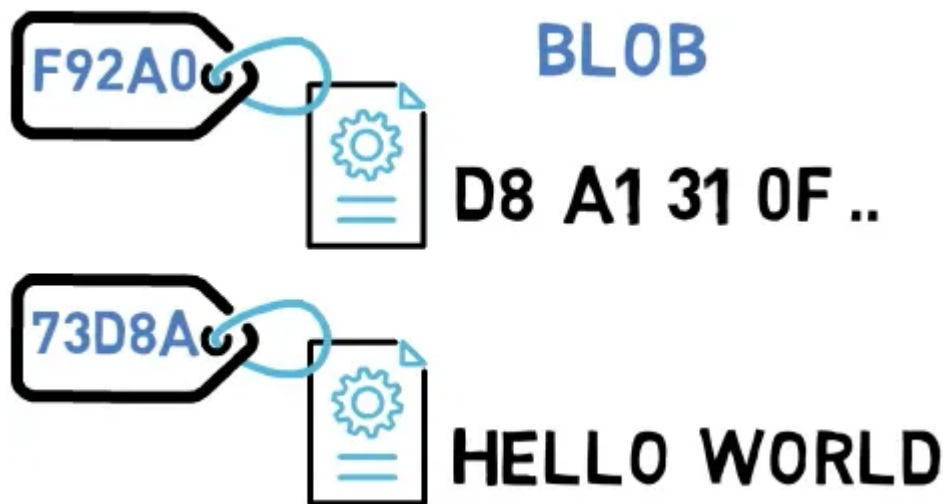
A file system begins with a *root directory* (in UNIX-based systems, `/`), which usually contains other directories (e.g., `/usr` or `/bin`). These directories contain other directories, and/or files (e.g., `/usr/1.txt`).

In `git`, the contents of files are stored in objects called **blobs**, binary large objects.

The difference between **blobs** and files is that files also contain meta-data. For example, a file “remembers” when it was created, so if you move that file into another directory, its creation time remains the same. **Blobs**, on the other hand, are

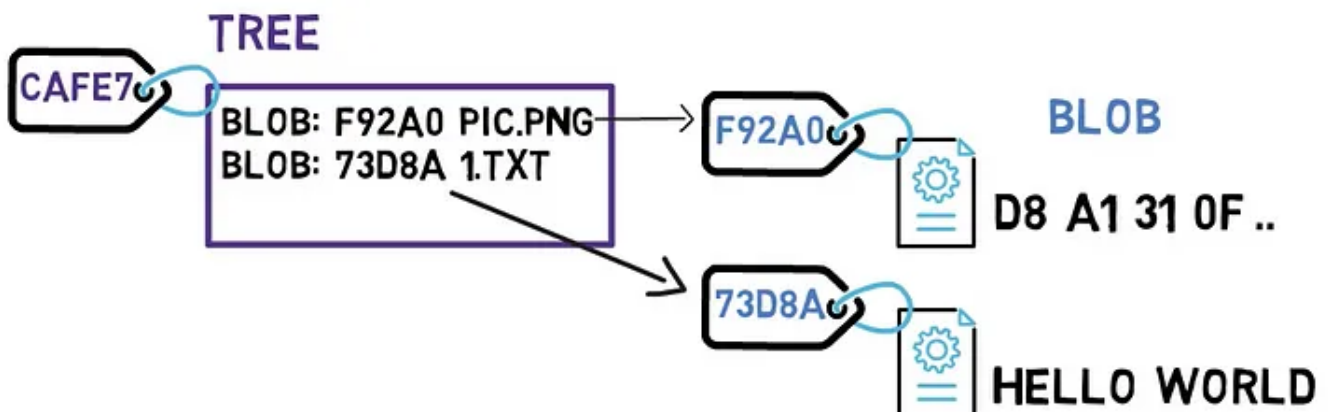
just contents — binary streams of data. A **blob** doesn't register its creation date, its name, or anything but its contents.

Every **blob** in `git` is identified by its SHA-1 hash. SHA-1 hashes consist of 20 bytes, usually represented by 40 characters in hexadecimal form. Throughout this post we will sometimes show just the first characters of that hash.



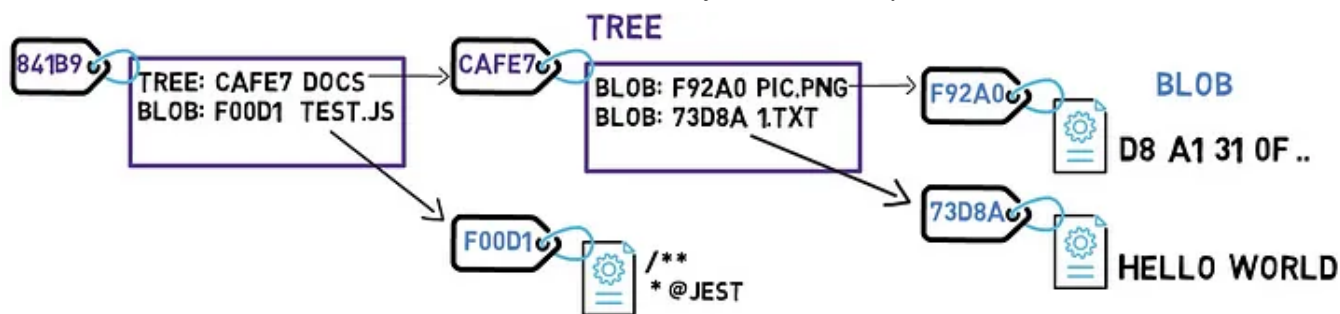
Blobs have SHA-1 hashes associated with them (Source: [Brief](#))

In `git`, the equivalent of a directory is a **tree**. A **tree** is basically a directory listing, referring to **blobs** as well as other **trees**. **Trees** are identified by their SHA-1 hashes as well. Referring to these objects, either **blobs** or other **trees**, happens via the SHA-1 hash of the objects.



A tree is a directory listing (Source: [Brief](#))

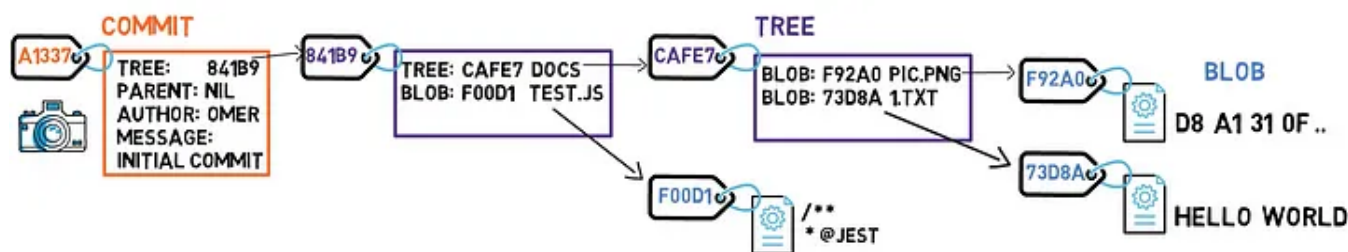
Note that the **tree** CAFE7 refers to the **blob** F92A0 as *pic.png*. In another **tree**, that same **blob** may have another name.



A tree may contain sub-trees, as well as blobs (Source: [Brief](#))

The diagram above is equivalent to a file system with a root directory that has one file at `/test.js`, and a directory named `/docs` with two files: `/docs/pic.png` and `/docs/1.txt`.

Now it's time to take a snapshot of that file system — and store all the files that existed at that time, along with their contents. In `git`, a snapshot is a **commit**. A **commit** object includes a pointer to the main **tree** (the root directory), as well as other meta-data such as the **committer**, a **commit** message and the **commit** time. In most cases, a **commit** also has one or more parent **commits** — the previous snapshot(s). Of course, **commit** objects are also identified by their SHA-1 hashes. These are the hashes we are used to seeing when we use `git log`.

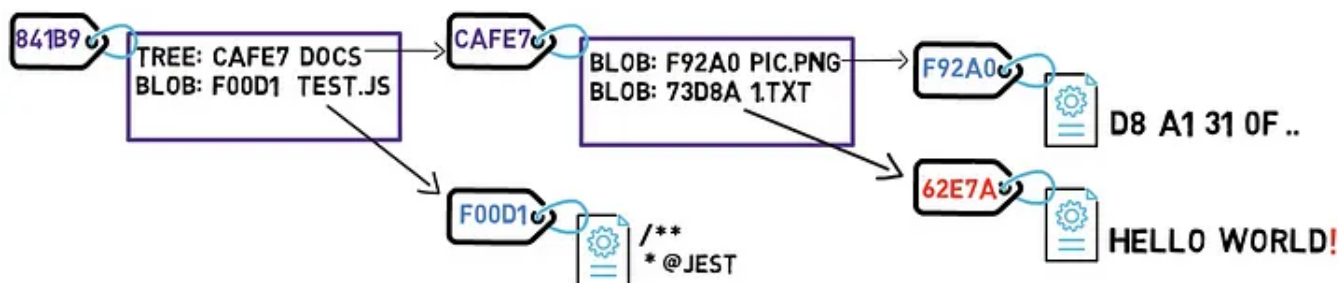


A commit is a snapshot in time. It refers to the root tree. As this is the first commit, it has no parent(s). (Source: [Brief](#))

Every **commit** holds the *entire snapshot*, not just diffs from the previous **commit(s)**.

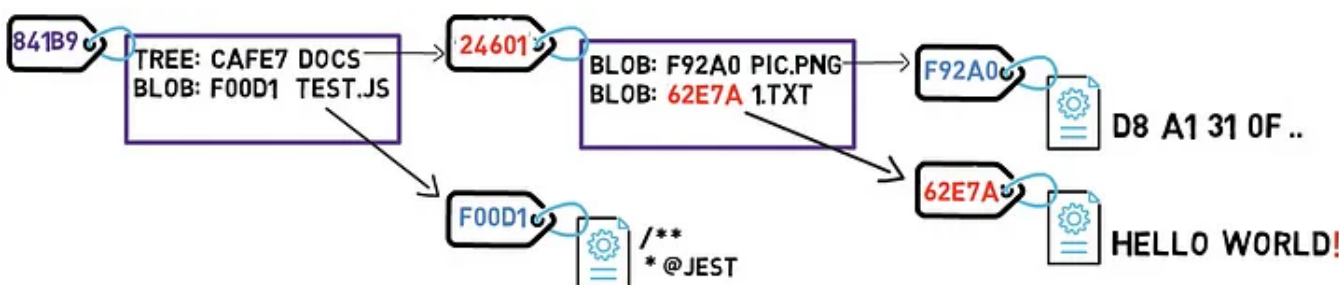
How can that work? Doesn't that mean that we have to store a lot of data every commit? Let's examine what happens if we change the contents of a file. Say that we edit `1.txt`, and add an exclamation mark — that is, we changed the content from `HELLO WORLD`, to `HELLO WORLD!`.

Well, this change would mean that we have a new **blob**, with a new SHA-1 hash. This makes sense, as `sha1("HELLO WORLD")` is different from `sha1("HELLO WORLD!")`.



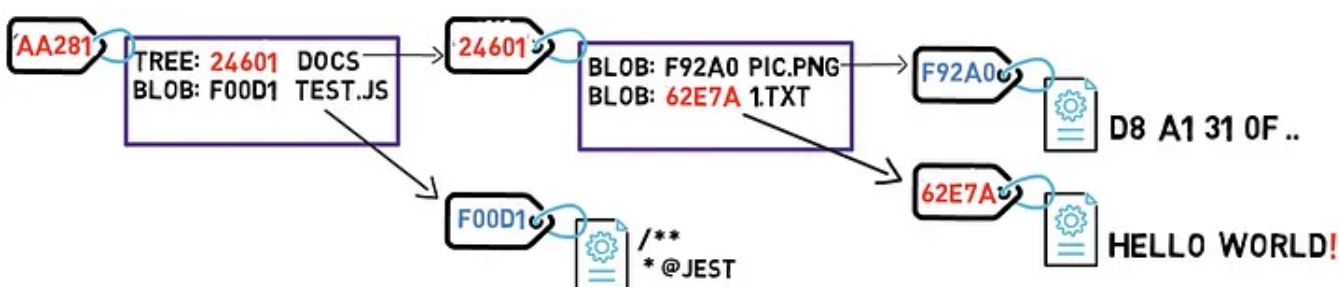
Changing the blob results in a new SHA-1 (Source: [Brief](#))

Since we have a new hash, then the **tree's** listing should also change. After all, our **tree** no longer points to **blob 73D8A**, but rather **blob 62E7A** instead. As we change the **tree's** contents, we also change its hash.



The tree that points to the changed blob needs to change as well (Source: [Brief](#))

And now, since the hash of that **tree** is different, we also need to change the parent **tree** — as the latter no longer points to **tree CAFE7**, but rather **tree 24601**. Consequently, the **parent tree** will also have a new hash.

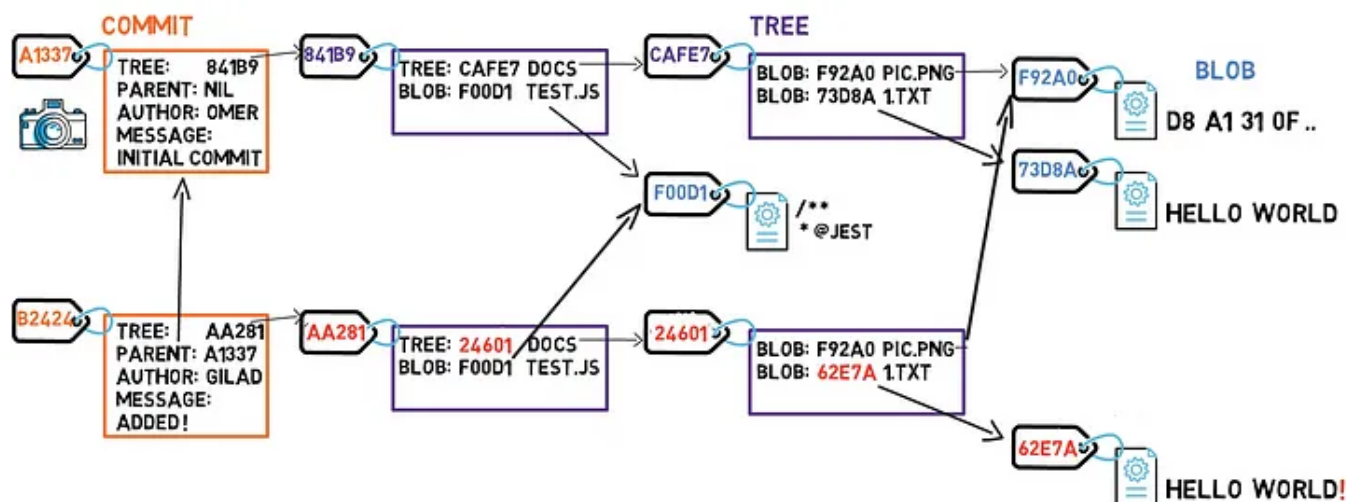


The root tree also changes, and so does its hash. (Source: [Brief](#))

Almost ready to create a new **commit** object, and it seems like we are going to store a lot of data — the entire file system, once more! But is that really necessary? Actually, some objects, specifically **blob** objects, haven't changed since the previous commit — **blob F92A0** remained intact, and so did **blob F00D1**.

So this is the trick — as long as an object doesn't change, we don't store it again. In this case, we don't need to store **blob F92A0** and **blob F00D1** once more. We only

refer to them by their hash values. We can then create our **commit** object.



Blobs that remained intact are referenced by their hash values (Source: [Brief](#))

Since this **commit** is not the first **commit**, it has a parent — **commit** A1337.

So to recap, we introduced three git objects:

- **blob** — contents of a file.
- **tree** — a directory listing (of **blobs** and **trees**).
- **commit** — a snapshot of the working tree.

Let us consider the hashes of these objects for a bit. Let's say I wrote the string `git is awesome!` and created a **blob** from it. You did the same on your system. Would we have the same hash?

The answer is — Yes. Since the **blobs** consist of the same data, they'll have the same SHA-1 values.

What if I made a **tree** that references the **blob** of `git is awesome!`, and gave it a specific name and metadata, and you did exactly the same on your system. Would we have the same hash?

Again, yes. Since the **trees** objects are the same, they would have the same hash.

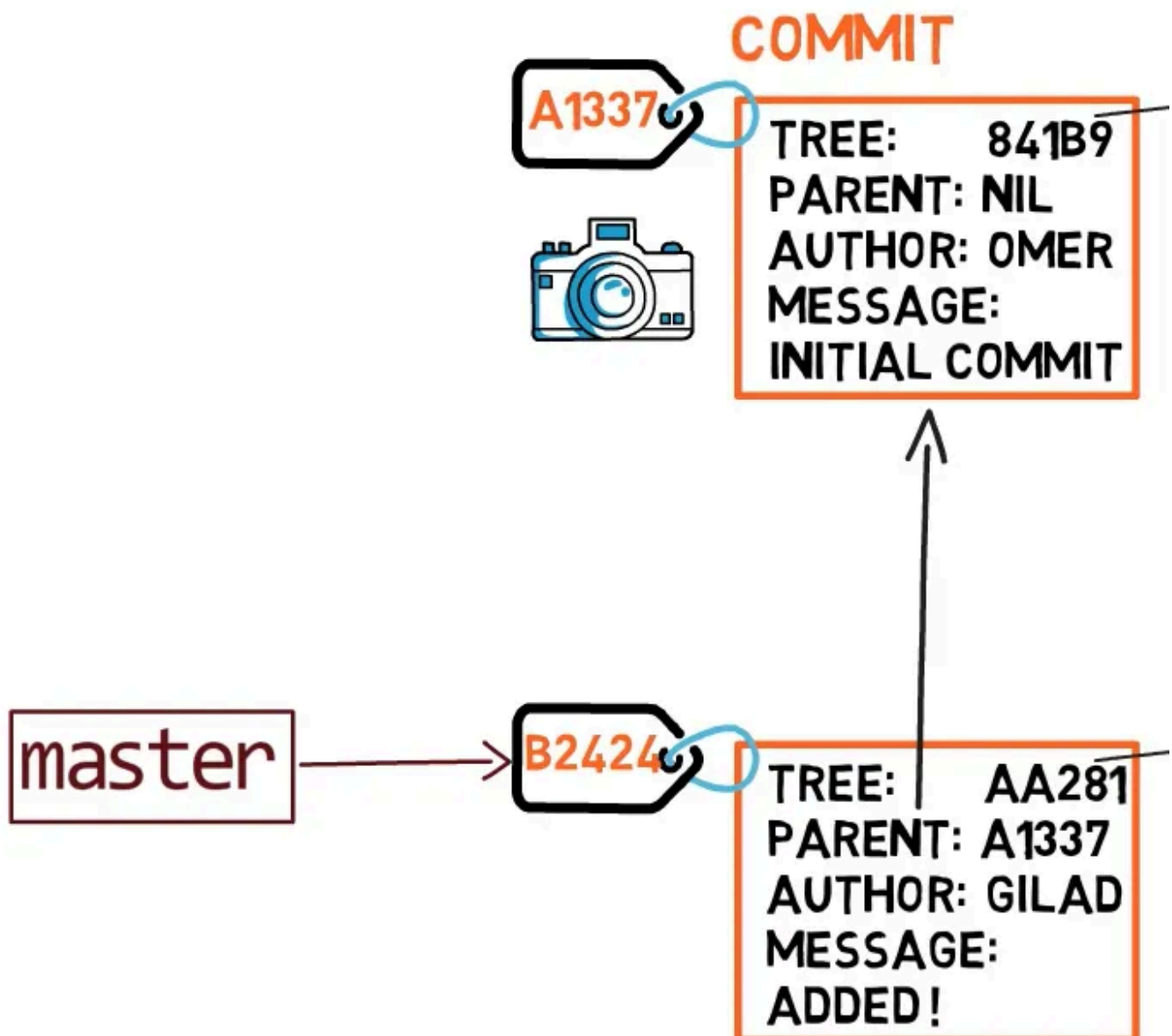
What if I created a **commit** of that **tree** with the commit message `Hello`, and you did the same on your system. Would we have the same hash?

In this case, the answer is — No. Even though our **commit** objects refer to the same **tree**, they have different **commit** details — time, committer etc.

Branches

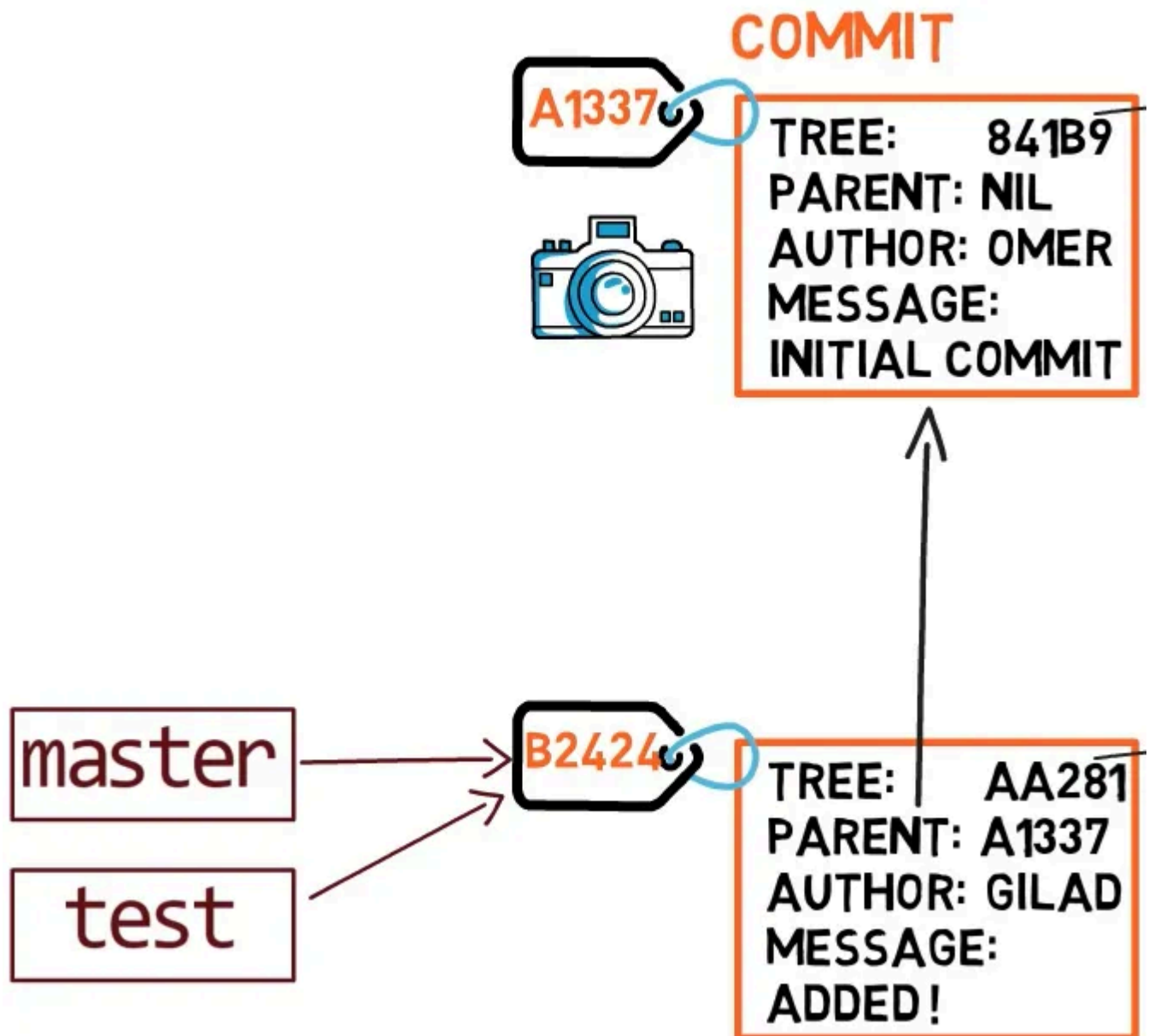
A branch is just a named reference to a commit.

We could always reference a **commit** by its SHA-1 hash, but humans usually prefer other forms to name objects. A **branch** is one way to reference a **commit**, but it's really just that. In most repositories, the main line of development is done in a branch called `master`. This is just a name, and it's created when we use `git init`, making it is widely used. However, it's by no means special, and we could use any other name we'd like. Typically, the branch points to the latest **commit** in the line of development we are currently working on.



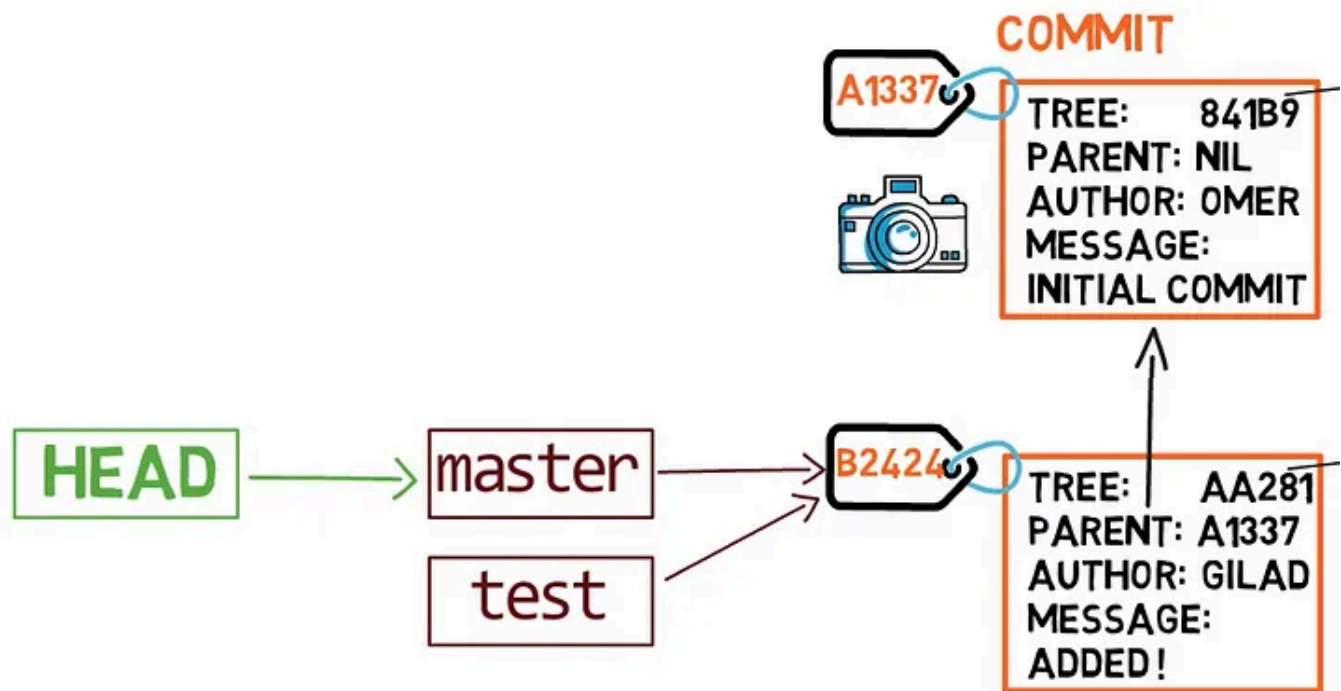
A branch is just a named reference to a commit (Source: [Brief](#))

To create another branch, we usually use the `git branch` command. By doing that, we actually create another pointer. So if we create a branch called `test`, by using `git branch test`, we are actually creating another pointer that points to the same **commit** as the branch we are currently on.



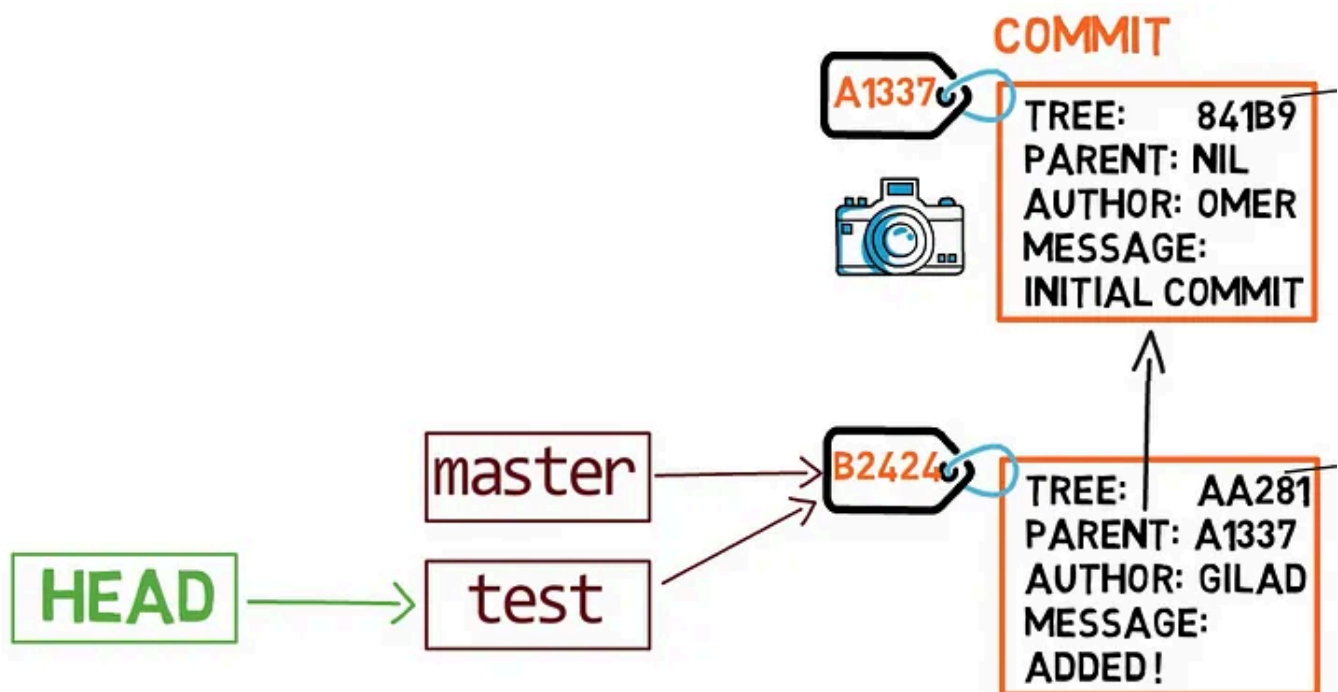
Using ``git branch`` creates another pointer (Source: [Brief](#))

How does `git` know what branch we're currently on? It keeps a special pointer called `HEAD`. Usually, `HEAD` points to a branch, which in turns points to a **commit**. In some cases, `HEAD` can also point to a **commit** directly, but we won't focus on that.



HEAD points to the branch we are currently on. (Source: [Brief](#))

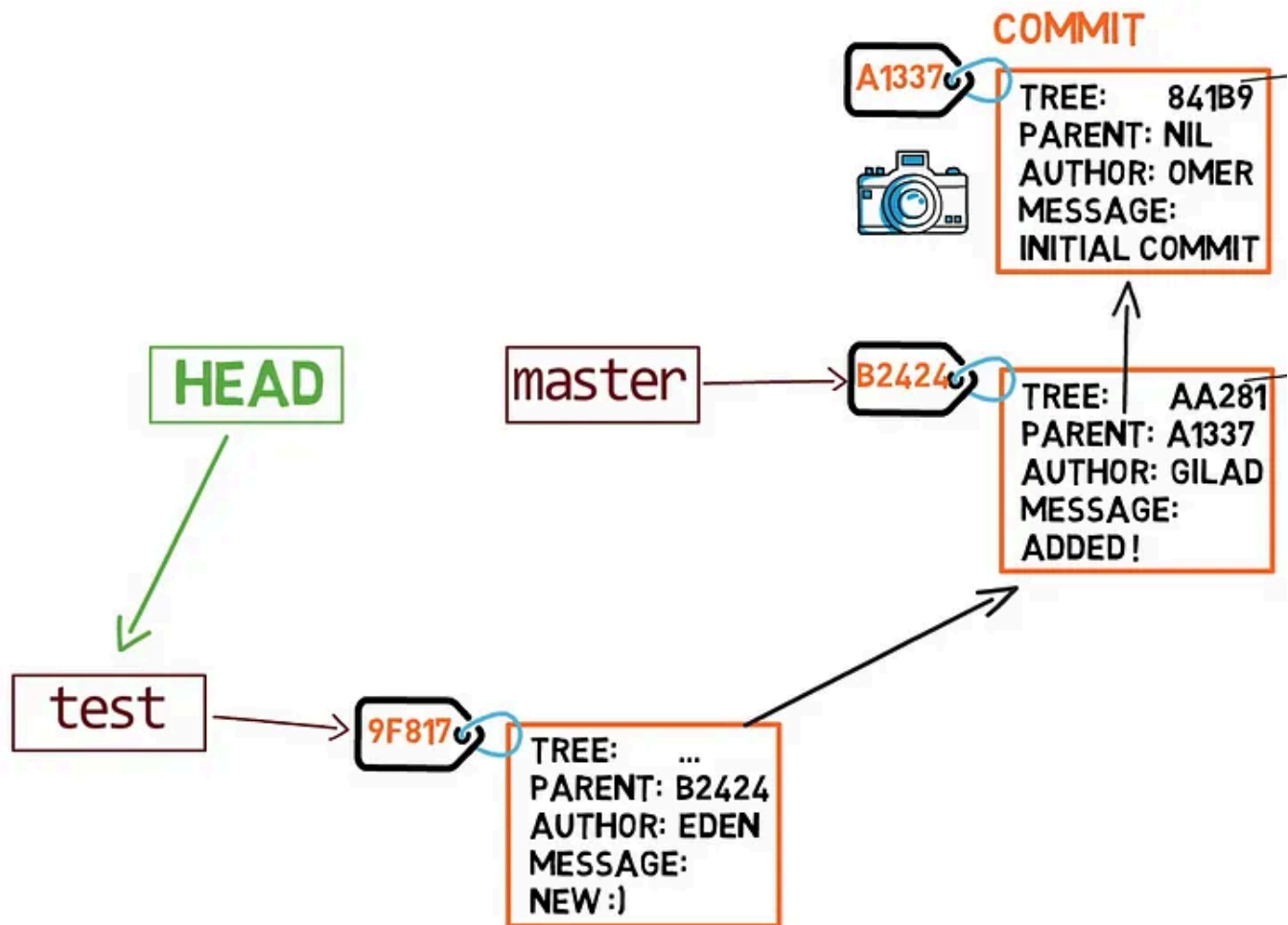
To switch the active branch to be `test`, we can use the command `git checkout test`. Now we can already guess what this command actually does — it just changes `HEAD` to point to `test`.



``git checkout test`` changes where ``HEAD`` points (Source: [Brief](#))

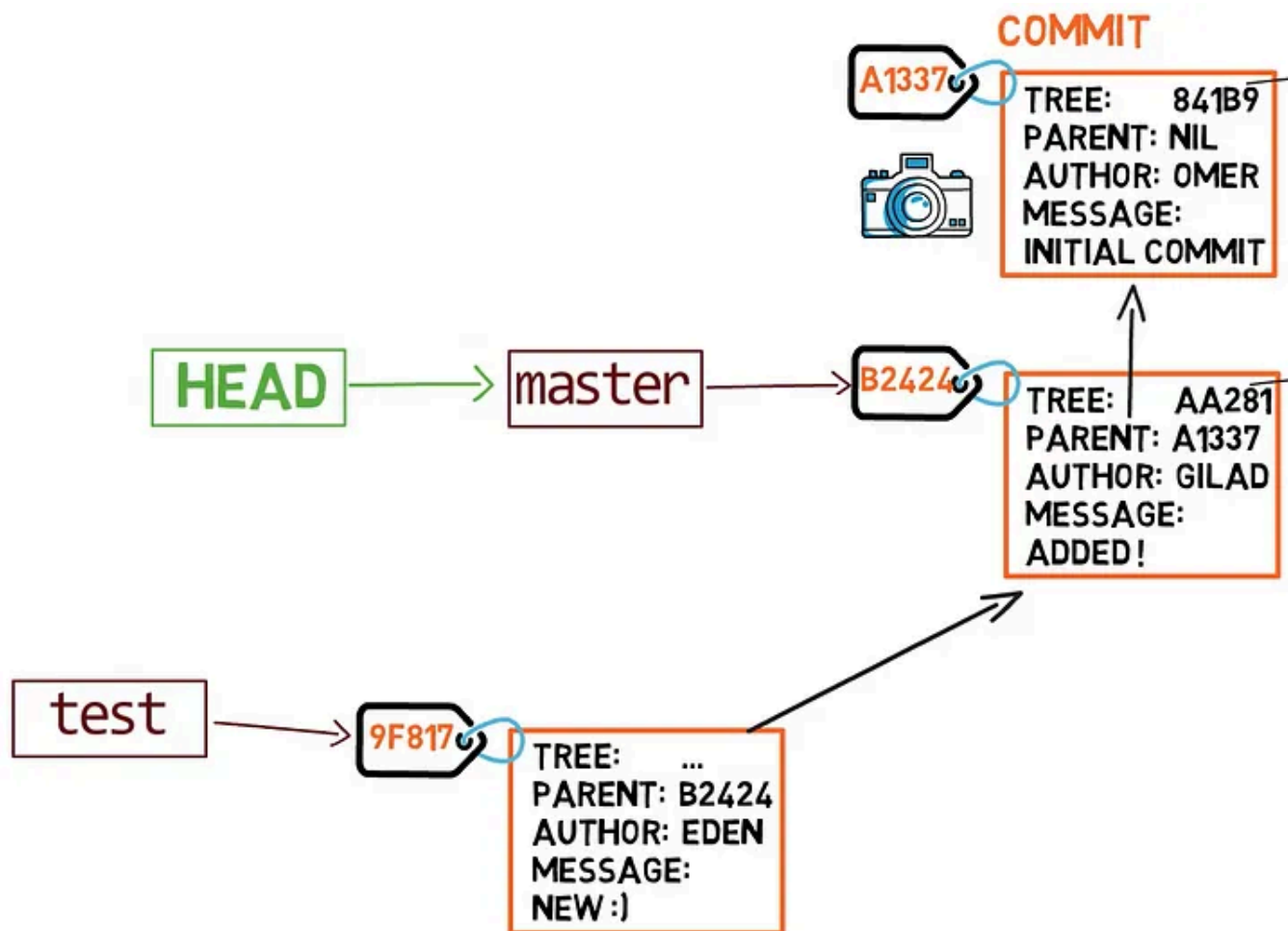
We could also use `git checkout -b test` before creating `test` branch, which is the equivalent of running `git branch test` to create the branch, and then `git checkout test` to move `HEAD` to point to the new branch.

What happens if we make some changes and create a new **commit** using `git commit`? Which branch will the new **commit** be added to? The answer is `test` branch, as this is the active branch (since `HEAD` points to it). Afterwards, `test` pointer will move to the newly added **commit**. Note that `HEAD` still points to `test`.



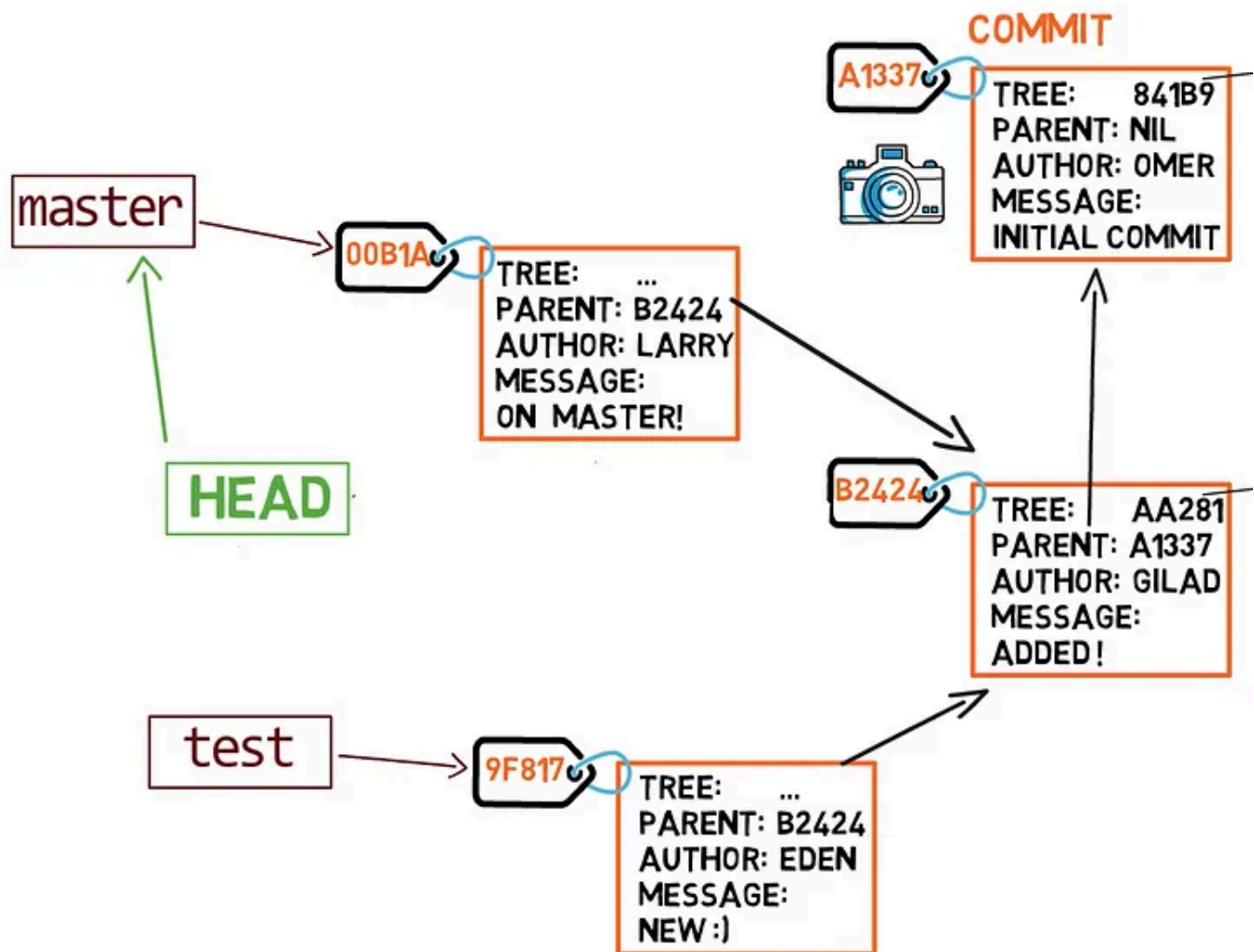
Every time we use ``git commit``, the branch pointer moves to the newly created commit. (Source: [Brief](#))

So if we go back to `master` by `git checkout master`, we move `HEAD` to point to `master` again.



(Source: [Brief](#))

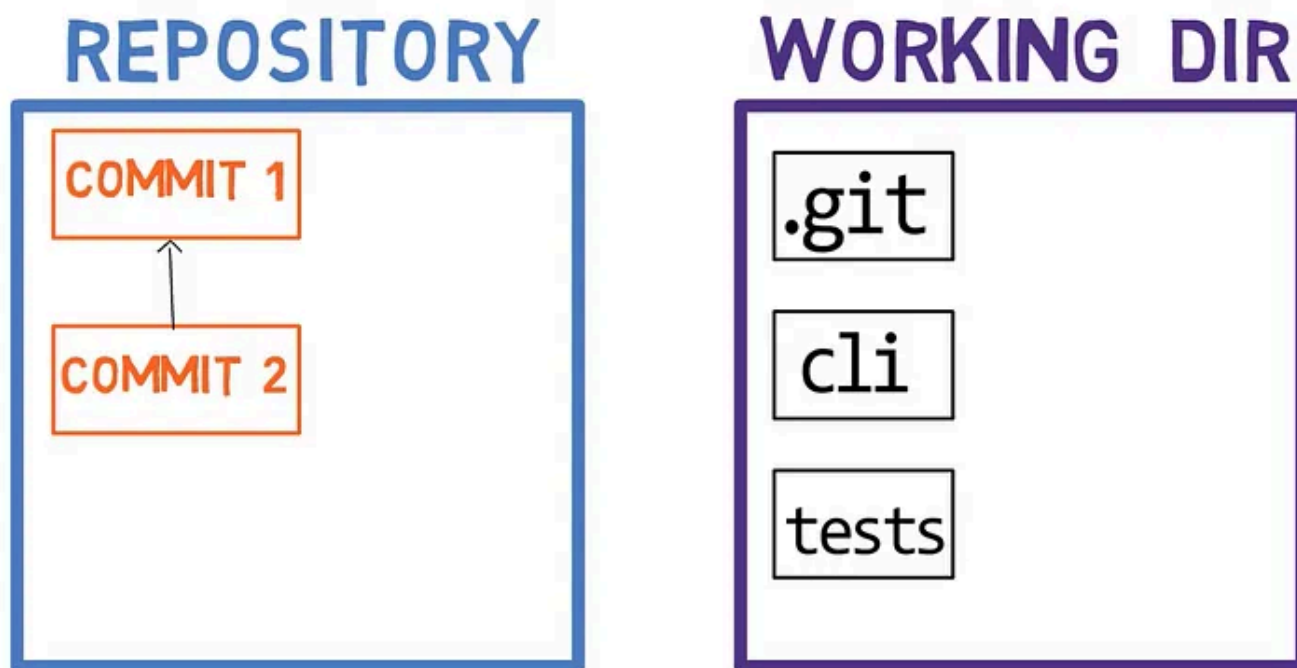
Now, if we create another **commit**, it will be added to **master** branch (and its parent would be **commit B2424**).

(Source: [Brief](#))

Recording Changes

Usually, when we work on our source code we work from a **working dir**. A **working dir(ectory)** (or **working tree**) is any directory on our file system which has a **repository** associated with it. It contains the folders and files of our project, and also a directory called `.git` that we will talk more about later.

After we make some changes, we want to record them in our **repository**. A **repository** (in short: **repo**) is a collection of **commits**, each of which is an archive of what the project's **working tree** looked like at a past date, whether on our machine or someone else's. A **repository** also includes things other than our code files, such as `HEAD`, branches etc.

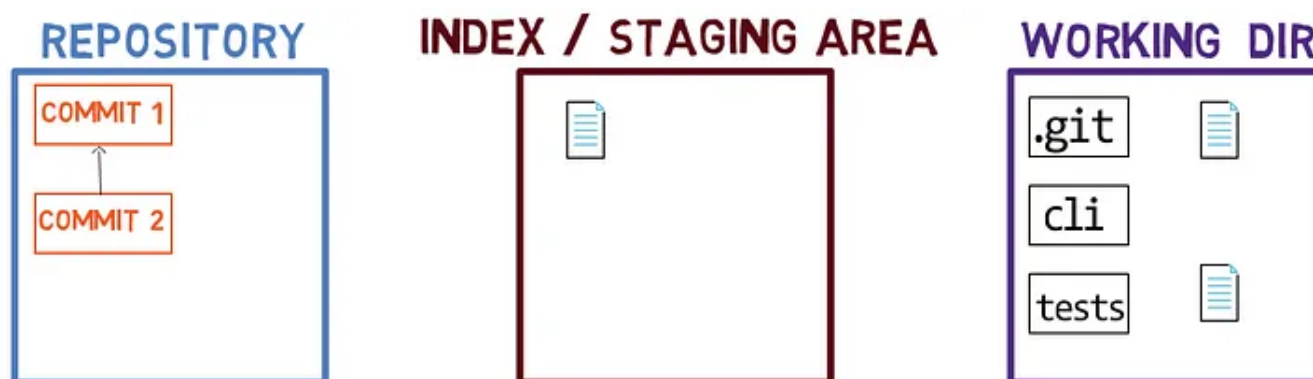


(Source: [Brief](#))

Unlike other, similar tools you may have used, `git` does not commit changes from the **working tree** directly into the **repository**. Instead, changes are first registered in something called the **index**, or the **staging area**. Both of these terms refer to the same thing, and they are used often in `git`'s documentation. We will use these terms interchangeably throughout this post.

When we `checkout` a branch, `git` populates the **index** with all the file contents that were last checked out into our **working directory** and what they looked like when they were originally checked out. When we use `git commit`, the **commit** is created based on the state of the **index**.

The use of the **index** allows us to carefully prepare each **commit**. For example, we may have two files with changes since our last **commit** in our **working dir**. We may only add one of them to the **index** (using `git add`), and then use `git commit` to record this change only.



(Source: [Brief](#))

Files in our **working directory** can be in one of two states: **tracked** or **untracked**.

Tracked files are files that `git` knows about. They either were in the last snapshot (**commit**), or they are **staged** now (that is, they are in the **staging area**).

Untracked files are everything else — any files in our **working directory** that were not in our last snapshot (**commit**) and are not in our **staging area**.

Creating a Repo — The Conventional Way

Let's make sure that we understand how the terms we've introduced relate to the process of creating a **repository**. This is just a quick high-level view. In [the next post](#), we will dive much deeper into this process.

- Note — most posts with shell commands show UNIX commands. I will provide commands for both Windows and UNIX, with screenshots from Windows, for the sake of variance. When the commands are exactly the same, I will provide them only once.

We will initialize a new **repository** using `git init repo_1`, and then change our directory to that of the repository using `cd repo_1`. By using `tree /f .git` we can see that running `git init` resulted in quite a few sub-directories inside `.git`. (The flag `/f` includes files in `tree`'s output).

```

C:\Users\omerr\swimminar>git init repo_1
Initialized empty Git repository in C:/Users/omerr/swimminar/repo_1/.git/

C:\Users\omerr\swimminar>cd repo_1

C:\Users\omerr\swimminar\repo_1>tree /f .git
Folder PATH listing for volume Windows
Volume serial number is 8E86-766C
C:\USERS\OMERR\SWIMMINAR\REPO_1\.GIT
├── config
│   ├── description
│   └── HEAD
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   ├── prepare-commit-msg.sample
│   └── update.sample
├── info
│   └── exclude
├── objects
│   ├── info
│   └── pack
└── refs
    ├── heads
    └── tags

```

Let us create a file inside `repo_1` directory:

```

C:\Users\omerr\swimminar\repo_1>echo hello > new_file.txt

C:\Users\omerr\swimminar\repo_1>dir /a
Volume in drive C is Windows
Volume Serial Number is 8E86-766C

Directory of C:\Users\omerr\swimminar\repo_1

04/18/2020  03:57 PM    <DIR>          .
04/18/2020  03:57 PM    <DIR>          ..
04/18/2020  03:52 PM    <DIR>          .git
04/18/2020  03:57 PM                8 new_file.txt
                1 File(s)                8 bytes
                3 Dir(s)  413,931,552,768 bytes free

```

On a Linux system:

```
$ echo hello > new_file.txt

$ ls -la
total 5
drwxr-xr-x 1 omerr 197609 0 Apr 18 16:00 ./
drwxr-xr-x 1 omerr 197609 0 Apr 18 15:52 ../
drwxr-xr-x 1 omerr 197609 0 Apr 18 15:52 .git/
-rw-r--r-- 1 omerr 197609 6 Apr 18 16:00 new_file.txt
```

This file is within our **working directory**. Yet, since we haven't added it to the **staging area**, it is currently **untracked**. Let us verify using `git status`:

```
C:\Users\omerr\swimminar\repo_1>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    new_file.txt

nothing added to commit but untracked files present (use "git add" to track)
```

The new file is untracked as we haven't added it to the staging area, and it wasn't included in a previous commit. (Source: [Brief](#))

We can now add this file to the **staging area** by using `git add new_file.txt`. We can verify that it has been staged by running `git status`:

```
C:\Users\omerr\swimminar\repo_1>git add new_file.txt

C:\Users\omerr\swimminar\repo_1>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   new_file.txt
```

Adding the new file to the staging area (Source: [Brief](#))

We can now create a **commit** using `git commit`:

```
C:\Users\omerr\swimminar\repo_1>git commit -m "First commit"
[master (root-commit) 8378b2d] First commit
1 file changed, 1 insertion(+)
create mode 100644 new_file.txt
```

Has something changed within `.git` directory? Let's run `tree /f .git` to check:

```
C:\Users\omerr\swimminar\repo_1>tree /f .git
Folder PATH listing for volume Windows
Volume serial number is 8E86-766C
C:\USERS\OMERR\SWIMMINAR\REPO_1\GIT
.
├── COMMIT_EDITMSG
├── config
├── description
├── HEAD
├── index
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   ├── prepare-commit-msg.sample
│   └── update.sample
├── info
│   └── exclude
├── logs
│   └── HEAD
│       └── refs
│           └── heads
│               └── master
├── objects
│   ├── 65
│   │   └── 578c66875f704ba442f7c6f2e699b5bfc203a0
│   ├── 83
│   │   └── 78b2d053a5a45526f34515b89189b2d636204e
│   ├── ce
│   │   └── 013625030ba8dba906f756967f9e9ca394464a
│   ├── f2
│   │   └── aa86d258996d05834897f0e3b668d342109bdc
│   ├── info
│   └── pack
├── refs
│   ├── heads
│   │   └── master
│   └── tags
└──
```

A lot of things have changed within `.git` (Source: [Brief](#))

Apparently, quite a lot has changed. On our next post, we will dive deeper into the structure of `.git` and understand what is going on under the hood when we run `git init`, `git add` OR `git commit`.

Summary

This post was our first about the internals of `git`. We started by covering the basic objects — **blobs**, **trees**, and **commits**. We said that a **blob** holds the contents of a file. A **tree** is a directory-listing, containing **blobs** and/or sub-trees. A **commit** is a snapshot of our working directory, with some meta-data such as the time or the commit message. We then discussed **branches** and explained that they are nothing but a named reference to a **commit**.

We went on to describe the **working directory**, a directory that has a repository associated with it, the **staging area (index)** which holds the **tree** for the next **commit**, and the **repository**, which is a collection of **commits**. We clarified how these terms relate to `git` commands we know by creating a new repository and committing a file using the well-known `git init`, `git add`, and `git commit`.

Hopefully, after following this post you feel you've deepened your understanding of what is happening under the hood when working with `git`.

Continue With Us to the Second Part

In our next post, we take it up a few levels, go much more hardcore, and create a **repository** without using commands such as `git init` OR `git commit` 💪. We will go further and create a new **branch**, switch to it and create a new **commit** on the new branch, all without using `git branch` OR `git checkout`. By following the next post, you will *really* understand what is going on under the hood and fully understand the answers to the questions posed at the beginning of this post. 😎

This post is a part of our Swimminars posts, and actually — together with the next post — they describe the first Swimminar ever. We plan to provide similar posts in the future, so please **comment** and let us know your thoughts or questions. You can also ask us to cover other tech topics you would like to know more about.

Ready for more? Continue reading part II.

Getting Hardcore — Creating a Repo From Scratch

A Hands-On Intro to Git Internals

medium.com

Omer Rosenbaum, Swimm's Chief Technology Officer. Cyber training expert and Founder of Checkpoint Security Academy. Author of Computer Networks (in Hebrew). Visit My YouTube Channel.

Additional References

A lot has been written and said about `git`. Specifically, I found these references to be useful:

- [Git Internals YouTube playlist — by Brief](#)
- [Tim Berglund's lecture — “Git From the Bits Up”](#)
- [Git from the Bottom Up — by John Wiegley](#)
- [as promised, docs: git for the confused](#)
- [Git Internals — Git Objects — from Pro Git book, by Scott Chacon and Ben Straub](#)

Engineering

Git

Git Internals

Computer Science

Tech

Open in app ↗



Search



Written by Omer Rosenbaum

51 Followers · Editor for Swimm

CTO @ [Swimm.io](#)

More from Omer Rosenbaum and Swimm



Omer Rosenbaum

Git UNDO—how to rewrite Git history with confidence

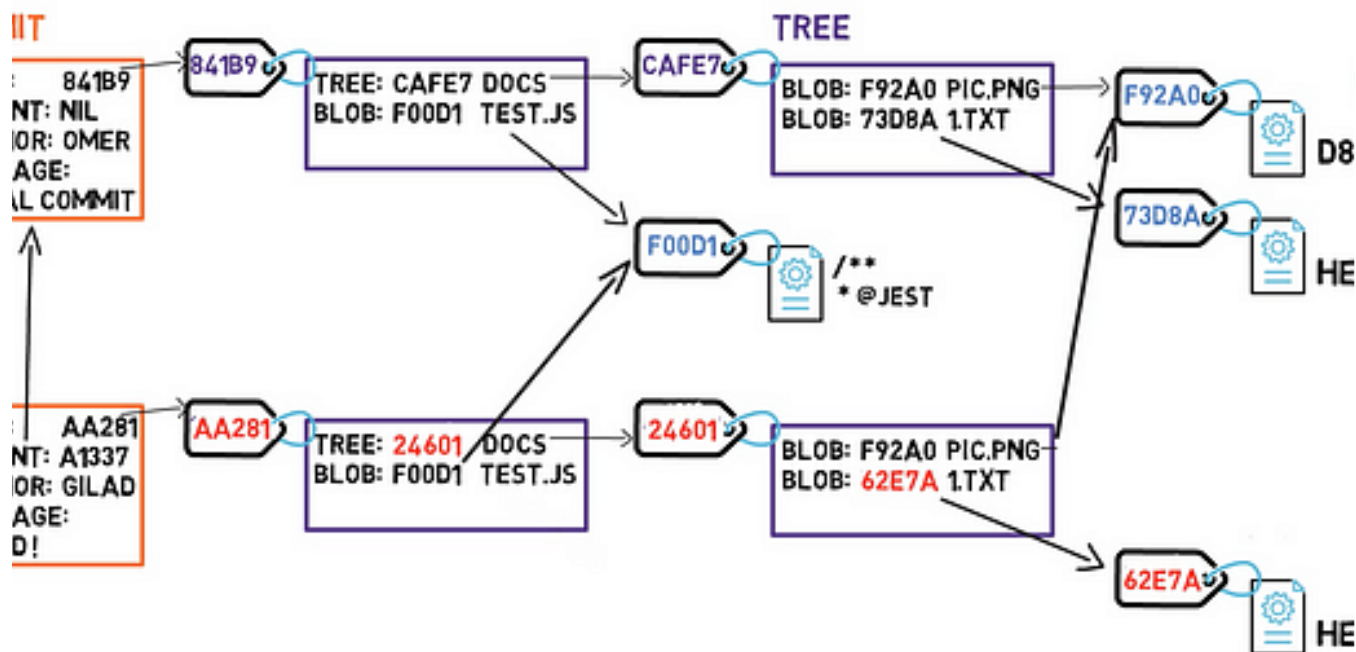
As a developer, your work with Git all the time.

18 min read · Apr 21, 2023



4





 Omer Rosenbaum in Swimm

Getting Hardcore—Creating a Repo From Scratch

A Hands-On Intro to Git Internals

13 min read · Apr 27, 2020

 70  2



 Omer Rosenbaum in Swimm

Professionalism by Forming a Team Habit

How does your workplace cultivate an environment of team development? How do you make sure your engineers have time for creative ideas and...

4 min read · Apr 19, 2020



the `Documentation/` directory of the Git source code. This file is a list of all the Git commands and their descriptions, and it is used to generate the Git man page (`git.1`) and to provide a list of commonly used Git commands in the `git help` output.

The `command-list.txt` file includes three columns:

1. Command name: This column lists the name of the Git command.
2. Description: This column provides a brief description of what the Git command does.
3. Common: This column indicates whether the command is commonly used or not. If a command is commonly used, the value is `yes`; if not, the value is `no`.

To add your new command to the `command-list.txt` file, follow these steps:

1. Open the `command-list.txt` file in a text editor.
2. Add a new line to the file for your new command. The line should be formatted as follows:

 Omer Rosenbaum

Will Copilot For Docs Replace Internal Documentation?

Generative AI and ChatGPT have created a lot of buzz, sparking discussions about how and if this technology can replace other tasks or...

6 min read · Apr 5, 2023



See all from Omer Rosenbaum

See all from Swimm

Recommended from Medium



Sourabh Kalal

Day56- Git Best Practices for DevOps Teams

In the fast-paced world of software development, Git stands as the cornerstone of version control systems. It's not just a tool for...

3 min read · Feb 28, 2024





fwsGonzo

An Introduction to Low-Latency Scripting for Game Engines, Part 2

Advanced examples, guest allocations, RPC

11 min read · May 22, 2024



83



Lists



Apple's Vision Pro

7 stories · 69 saves



Stories to Help You Grow as a Software Developer

19 stories · 1093 saves



Good Product Thinking

11 stories · 586 saves



Leadership

50 stories · 341 saves



 Alexander obidiegwu

50 Coding Laws That Would Make You A Decent Programmer.

Follow these laws or get fired.

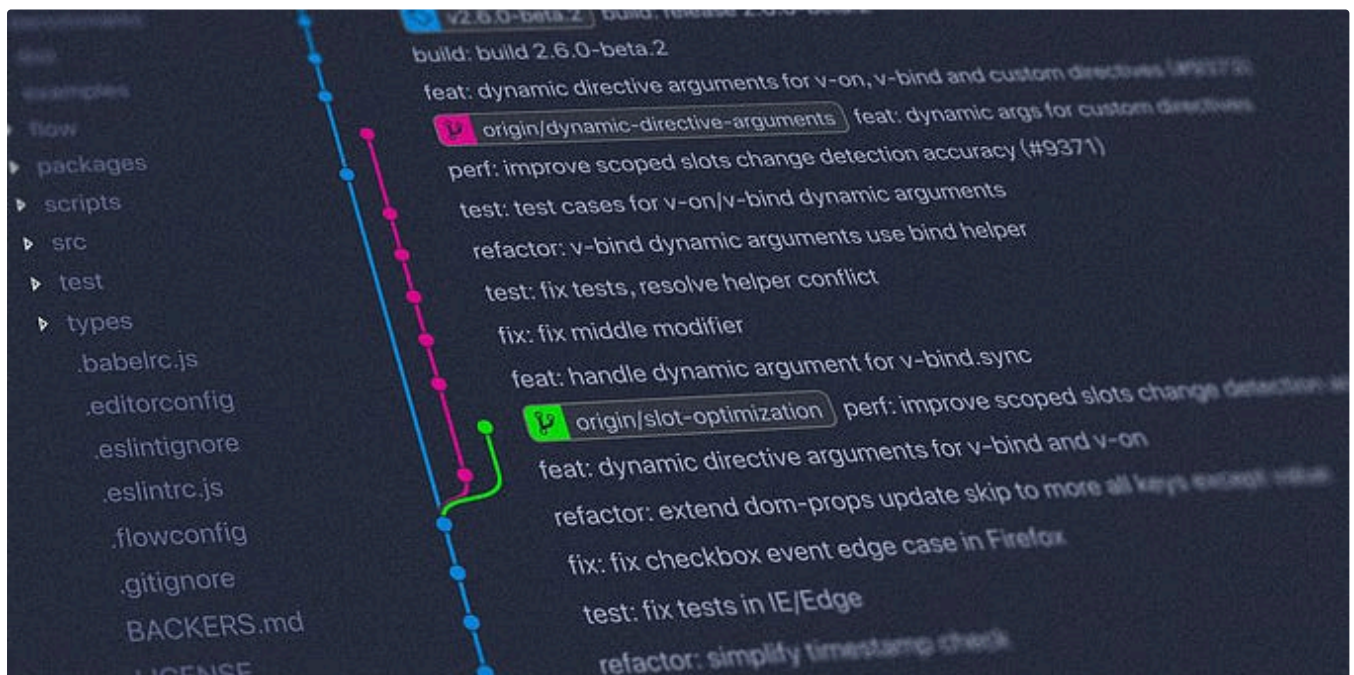
24 min read · Mar 2, 2024



1.7K



24



 Brian Jenney

Git Doesn't Have to be Complicated. Here's my Dead Simple Workflow for you to Steal

Let's explore the dead simple Git workflow I've been using for years to commit my work, debug complicated issues and save a few release...

6 min read · 2 days ago



 Benoit Ruiz in Better Programming

Advice From a Software Engineer With 8 Years of Experience

Practical tips for those who want to advance in their careers

22 min read · Mar 20, 2023





 Alexander Nguyen in Level Up Coding

Why I Keep Failing Candidates During Google Interviews...

They don't meet the bar.

4 min read · Apr 13, 2023



6.7K



198



See more recommendations